NAZON Tracking Mastery Guide

A Blueprint for Building Lusaka's Premier Driver Advisory & Fleet Intelligence Platform

Navigate safely. Track intelligently. Prevent misuse.

Version: 1.0
Date: 12 May 2026
Target Audience: Founders, Junior Developers, and Product Builders

---

How to Use This Guide

Welcome to the NAZON Tracking roadmap. This document is not a theoretical manual; it is a sequential, build-as-you-learn blueprint. As a builder, your goal is to construct a dual-purpose platform: a mobile app that helps drivers safely navigate Lusaka's notorious peak-hour congestion, and a robust fleet intelligence backend that empowers NGOs, government agencies, and fleet companies to monitor vehicles and prevent misuse.

Read sequentially. Build iteratively. Do not skip to advanced hardware integrations before your foundational trip-logging API is secure. Treat this guide as your mentor.

Table of Contents

- **BEGINNER PHASE: Foundations & Core Logic**
  - L01 Vision, Users, and Problem Definition
  - L02 Safety, Congestion, and Misuse Use Cases
  - L03 GPS, Maps, and Route Basics

- L04 Wireframes and User Journeys
- L05 Time-Based Warnings and Simple Rules
- L06 Basic Routing Logic
- L07 Trip Logging Fundamentals
- L08 Backend Foundations and System Overview

- **INTERMEDIATE PHASE: Fleet Controls & Live Data**
  - L09 Auth, Roles, and Company Accounts
  - L10 Vehicle-Driver Assignment and Trip Ownership
  - L11 Geofencing and Restricted Zones
  - L12 Live Traffic and Map Integrations
  - L13 Driver Behavior Monitoring and Alerts
  - L14 Manager Dashboard v1

- **PRO PHASE: Intelligence & Scale**
  - L15 Risk Scoring Engine
  - L16 Smart Routing and Recommendation Engine
  - L17 Real-Time Alerts, Notifications, and SOS
  - L18 Analytics, Reporting, and Operational Insights
  - L19 Hardware/Device Integrations and Telematics
  - L20 Final Architecture, Deployment, and v1 Capstone

BEGINNER

Phase 1: Foundations & Core Logic

The Beginner Phase establishes the fundamental architecture of the NAZON Tracking platform. You will define the problem space, map out user roles, and build the initial components required to track a device and store trip data securely.

L01 — Vision, Users, and Problem Definition

1) Why this matters: You cannot build a solution without clearly defining who you are building for. A consumer app requires different constraints than an enterprise fleet tool.

2) Core ideas: Define the ordinary driver (needs safety/routing), the fleet driver (needs assignment/logging), and the fleet manager (needs oversight/misuse prevention).

3) Practical example: A driver unfamiliar with Lusaka needs to get from Cairo Road to Kabulonga at 17:30. Simultaneously, their NGO fleet manager needs to ensure the vehicle isn't used for personal errands later that night.

4) Build output: A written Product Requirements Document (PRD) outline detailing the three user personas.

5) Practice tasks: Draft 3 user stories for each persona focusing on their primary pain points in Lusaka.

L02 — Safety, Congestion, and Misuse Use Cases

1) Why this matters: App logic must mirror real-world business and safety rules to be valuable.

2) Core ideas: Categorize road conditions (peak-hour stress, night risks) and fleet violations (after-hours usage, route deviation, unauthorized stops).

> **WARNING: "Unsafe roads at night" must be based on verified incident reports, trusted internal company policies, manager-defined restricted areas, or official risk intelligence. Never rely on community rumors or unverified claims, which can create legal liability and bias.**

3) Practical example: Defining a rule that flags an NGO vehicle if the ignition is on past 20:00 without prior authorization.

4) Build output: A JSON schema defining rule types (e.g., `TIME_RESTRICTION`, `ZONE_RESTRICTION`).

5) Practice tasks: Map out the exact logic flow when a vehicle breaks a curfew rule.

L03 — GPS, Maps, and Route Basics

1) Why this matters: Location data is the lifeblood of NAZON Tracking.

2) Core ideas: Latitude/Longitude formatting, requesting device location permissions, rendering a basic map view, and calculating distance.

3) Practical example: Plotting a standard route from Kenneth Kaunda International Airport into the CBD.

4) Build output: A basic mobile view displaying a map with a blue dot for the user's current location.

5) Practice tasks: Implement location permission handling gracefully in your mobile framework.

L04 — Wireframes and User Journeys

1) Why this matters: Visualizing the flow prevents wasted engineering effort on backend APIs that won't be used.

2) Core ideas: App navigation, dashboard layout, and alert visibility.

3) Practical example: Sketching a "Start Shift" screen for fleet drivers vs. a generic "Where to?" screen for ordinary drivers.

4) Build output: Low-fidelity mockups of the mobile app and the manager dashboard.

5) Practice tasks: Map the screen flow for a driver receiving a "Congestion Ahead" alert.

L05 — Time-Based Warnings and Simple Rules

1) Why this matters: The simplest form of intelligence is time-based logic, serving as the foundation for the risk engine.

2) Core ideas: Evaluating current time against a set of predefined rules.

3) Practical example: A simple Python function that returns a warning if a driver attempts to navigate through a known restricted zone after 19:00.

4) Build output: A basic rule-evaluation script.

```
def evaluate_time_rule(current_time, route_zone):
    if route_zone == "RESTRICTED_CBD" and current_time.hour >= 19:
         return {"status": "WARNING", "message": "Avoid this route at night
based on company policy."}
    return {"status": "OK"}
```

5) Practice tasks: Write a unit test verifying that 18:59 passes and 19:00 fails.

L06 — Basic Routing Logic

1) Why this matters: Users need options. The fastest route is not always the safest or the most compliant.

2) Core ideas: Generating multiple route arrays: fastest vs. safer vs. company-approved.

3) Practical example: Offering a route that avoids heavy traffic on Great East Road during 17:00 rush hour, even if the distance is slightly longer.

4) Build output: A mock routing response returning two distinct path options.

5) Practice tasks: Implement a UI toggle to switch between "Fastest" and "Company Approved" routes.

L07 — Trip Logging Fundamentals

1) Why this matters: Without historical trip data, managers cannot audit usage or investigate misuse.

2) Core ideas: Capturing start/end coordinates, timestamps, and route breadcrumbs.

3) Practical example: A fleet driver finishes their delivery; the app packages the trip data and prepares to send it to the server.

4) Build output: A JSON payload representing a completed trip.

```
{
  "trip_id": "trp_1048",
  "driver_id": "drv_099",
  "vehicle_id": "ZAM_1234",
  "start_time": "2023-10-12T08:00:00Z",
  "end_time": "2023-10-12T08:45:00Z",
  "distance_km": 14.2
}
```

5) Practice tasks: Validate this payload structure using a basic schema validator.

L08 — Backend Foundations and System Overview

1) Why this matters: A reliable tech stack is required to handle concurrent vehicle tracking and high-frequency location updates.

2) Core ideas: Setting up FastAPI for the API, PostgreSQL/PostGIS for spatial data, and Redis for caching live locations.

3) Practical example: Establishing the database schema to handle ordinary users and organizational fleets separately.

4) Build output: A running local FastAPI server with a `/health` endpoint.

5) Practice tasks: Configure a basic PostgreSQL database and connect it to your FastAPI app.

> **Phase 1 Recap:** You have defined the users, established safe routing rules, designed the interfaces, and built the foundational backend structure. You are now ready to handle live data and fleet mechanics.

INTERMEDIATE Phase 2: Fleet Controls & Live Data

In this phase, NAZON Tracking evolves from a basic map app into an operational fleet tool. We introduce organizational constraints, real-time spatial rules, and dashboard visibility.

L09 — Auth, Roles, and Company Accounts

1) Why this matters: Security and data isolation. An NGO manager should only see their own vehicles.

2) Core ideas: JWT authentication, Role-Based Access Control (RBAC), and multi-tenancy (company accounts).

3) Practical example: A user logs in. The server checks if they are a `DRIVER` or an `ADMIN_MANAGER` and returns the appropriate dashboard data.

4) Build output: Secure login endpoints and middleware to verify user roles.

5) Practice tasks: Implement a route that returns HTTP 403 Forbidden if a driver tries to access a manager endpoint.

L10 — Vehicle-Driver Assignment and Trip Ownership

1) Why this matters: To prevent misuse, you must establish an audit trail of *who* is driving *what* vehicle at *any* given time.

2) Core ideas: Shift check-in/check-out flows, assigning specific drivers to database vehicle entities.

3) Practical example: A government ministry driver scans a QR code on the dashboard to bind their session to a specific Toyota Hilux for the day.

4) Build output: API endpoints for `/assign-vehicle` and `/release-vehicle`.

5) Practice tasks: Build logic to prevent a vehicle from being assigned to two different drivers simultaneously.

L11 — Geofencing and Restricted Zones

1) Why this matters: Geofencing automates misuse detection by identifying when vehicles leave authorized boundaries.

2) Core ideas: Using PostGIS to define polygons (geofences) and performing point-in-polygon checks.

3) Practical example: Drawing a virtual boundary around the Lusaka industrial area. If an NGO vehicle exits this zone during working hours without a logged destination, an alert is triggered.

4) Build output: A database table for `Geofences` and a spatial query function to check location containment.

5) Practice tasks: Write a PostGIS SQL query using `ST_Contains` or `ST_Within`.

L12 — Live Traffic and Map Integrations

1) Why this matters: Advisory features rely on accurate, real-world data.

2) Core ideas: Integrating third-party map APIs (e.g., TomTom, Mapbox, Google Maps) to fetch live congestion layers and incident reports.

3) Practical example: Querying a traffic API to see if Church Road is congested, and if so, dynamically recalculating the ETA.

4) Build output: A backend service that proxies traffic API requests and formats them for the mobile client.

5) Practice tasks: Parse a traffic API JSON response and extract the congestion level (e.g., heavy, moderate).

L13 — Driver Behavior Monitoring and Alerts

1) Why this matters: Safety and asset protection require identifying dangerous or wasteful driving habits.

2) Core ideas: Calculating speed from location points, detecting route deviations, and tracking excessive idle times.

3) Practical example: A company vehicle sits idle with the engine running for 45 minutes; the system logs an "Excessive Idle" event to prevent fuel theft or waste.

4) Build output: A background worker (e.g., Celery) that evaluates incoming location streams for behavioral anomalies.

5) Practice tasks: Write an algorithm to calculate speed given two GPS coordinates and a timestamp difference.

L14 — Manager Dashboard v1

1) Why this matters: Fleet managers need a centralized command center to view operations in real-time.

2) Core ideas: Web UI development, data tables, and live map plotting for active vehicles.

3) Practical example: A fleet manager logs into a web portal and sees a live map of all 15 active vehicles navigating Lusaka, with one blinking red due to a geofence violation.

4) Build output: A React/Next.js dashboard consuming the APIs built in previous lessons.

5) Practice tasks: Implement a data grid showing driver name, vehicle ID, current status, and active alerts.

> **Phase 2 Recap:** You have successfully integrated multi-tenant fleet controls, established spatial boundaries via geofencing, and built out behavior monitoring. The system can now observe and report on active operations.

PRO Phase 3: Intelligence & Scale

The final phase transitions the platform from a reactive tracking system to a proactive intelligence engine. We will implement risk scoring, WebSockets for live data, and prepare for production deployment.

L15 — Risk Scoring Engine

1) Why this matters: Raw data is overwhelming. Managers need synthesized metrics to evaluate driver performance at a glance.

2) Core ideas: Aggregating behavior events, time-of-day factors, and route risk into a standardized numerical score (e.g., 0-100).

3) Practical example: A driver who frequently speeds and travels through unverified/restricted zones at night receives a safety score of 45/100, prompting a management review.

4) Build output: A cron job or scheduled task that calculates and updates driver risk scores daily.

5) Practice tasks: Define the weighting formula for different violation types (e.g., route deviation = -5 pts, speeding = -10 pts).

L16 — Smart Routing and Recommendation Engine

1) Why this matters: True advisory platforms suggest optimal actions rather than just displaying data.

2) Core ideas: Combining third-party traffic data with historical fleet data and risk scores to rank route options.

3) Practical example: The engine recommends Route B over Route A during peak hours, knowing that Route A historically results in 20% higher idle times based on fleet records.

4) Build output: An advanced `/routes/recommend` endpoint returning fully scored path options.

5) Practice tasks: Integrate a caching layer (Redis) to store route calculations for frequently requested paths.

L17 — Real-Time Alerts, Notifications, and SOS

1) Why this matters: Critical incidents require immediate attention, not end-of-day reports.

2) Core ideas: Implementing WebSockets for instant dashboard updates and Push Notifications for mobile clients. Adding an SOS workflow.

3) Practical example: A driver presses the in-app SOS button; the manager dashboard immediately triggers an audible alarm and centers the map on the vehicle's location.

4) Build output: A WebSocket server broadcasting alert events.

5) Practice tasks: Create a basic WebSocket client script to listen for incoming `URGENT_ALERT` messages.

L18 — Analytics, Reporting, and Operational Insights

1) Why this matters: Long-term value for fleet organizations comes from actionable business intelligence (reducing fuel costs, optimizing maintenance).

2) Core ideas: Data aggregation, generating PDF/CSV reports, and charting utilization trends.

3) Practical example: Generating an end-of-month report showing total kilometers driven, fuel waste due to idling, and instances of unauthorized private errands for a specific department.

4) Build output: A reporting module in the dashboard with date-range filters and export capabilities.

5) Practice tasks: Write a SQL query grouping total distance traveled by vehicle per month.

L19 — Hardware/Device Integrations and Telematics

1) Why this matters: Phones can be turned off. Hardwired telematics provide undeniable ground truth.

2) Core ideas: Expanding the API to ingest data from physical OBD-II trackers or hardwired GPS modules alongside mobile app data.

3) Practical example: Processing a TCP/UDP stream from a hardwired vehicle tracker to verify ignition status independently of the driver's phone.

4) Build output: A standardized ingestion pipeline that normalizes data from both mobile apps and physical trackers into a single format.

5) Practice tasks: Define a data model that accommodates hardware-specific metrics like Engine RPM or Fuel Level.

L20 — Final Architecture, Deployment, and v1 Capstone

1) Why this matters: Code on a local machine serves no users. The system must be securely deployed and scalable.

2) Core ideas: Containerization (Docker), load balancing, CI/CD pipelines, and production security audits.

3) Practical example: Deploying the FastAPI backend, PostGIS database, and Redis cache via Docker Compose to a secure cloud environment.

4) Build output: The live NAZON Tracking v1 platform.

5) Practice tasks: Write a `Dockerfile` and a `docker-compose.yml` file for the entire stack.

NAZON Tracking v1 — Full Working Platform

By completing this 20-lesson roadmap, you have engineered a comprehensive system capable of transforming transportation intelligence in Lusaka. The v1 platform successfully integrates:

- A driver-facing mobile application (built in Flutter/React Native) delivering context-aware routing, congestion avoidance, and time-based safety advisories based strictly on verified parameters.
- A secure, multi-tenant API backend (FastAPI/PostgreSQL) capable of handling concurrent spatial operations and telemetry data.
- A real-time manager dashboard enabling NGOs and corporate fleets to enforce accountability, manage geofences, and receive instantaneous operational alerts regarding vehicle misuse.

Next Steps After v1

Once v1 is stable and in production, the roadmap expands into predictive analytics. Future iterations should explore machine learning models to predict traffic patterns before they occur, deeper OEM telematics integrations for predictive vehicle maintenance, and automated compliance reporting tailored to government and NGO auditing standards.

Keep building. Stay disciplined. Measure everything.